



Since these are big gaps. We need a full rewrite of the plan including all roles now established and the new research and planning from each perspective persona you will assume.

So let's start with a grande new plan write up then once approved we will look at code

I now have everything. Harvey just added 110+ new legal data sources in March 2026 and GPT-5.2 in January — the competitive window is narrowing. Let me write the definitive plan.

LexCore v4.0

North America's Legal Intelligence Database

The Complete Master Specification

StackConsulting | Built with the Agentic Horde | Approved for Code Execution

PART I — STRATEGIC FOUNDATION

1.1 CEO Perspective: Vision, Moat & Urgency

Harvey is accelerating. January 2026: GPT-5.2 integration. March 2026: 110+ new legal data sources added, Deep Analysis in Custom Workflow Agents live. April 2026: expanded automation and integrations shipping. They are not standing still and the window to establish a competing data moat is narrowing every quarter.^{[1] [2] [3]}

The CEO's mandate is clear: LexCore does not compete with Harvey feature-for-feature. LexCore makes Harvey strategically irrelevant for a specific, underserved market segment, then expands from that beachhead.

The founding insight Harvey cannot counter: Harvey is a US corporation. Its infrastructure — regardless of what data center region it uses — is subject to the US CLOUD Act (18 U.S.C. § 2703). A US court can compel Harvey to disclose Canadian client query data regardless of where it is physically hosted. Canadian law firms handling privileged matter data cannot safely use Harvey under PIPEDA Principle 4.1.3's adequacy standard. LexCore, built as a Canadian-incorporated entity running on Canadian-owned infrastructure, is the only legal intelligence platform that can offer true data sovereignty to Canadian legal professionals.^[4]
^[5]

This is not a feature. This is an existential structural advantage Harvey cannot replicate without restructuring its corporate identity.

LexCore's Three-Layer Moat (in order of defensibility):

- **Layer 1 — Data Sovereignty:** Canadian incorporation + Canadian-hosted infrastructure + zero US corporate parent = the only truly PIPEDA-sovereign legal AI platform on the continent^[5]
- **Layer 2 — Cross-Border Coverage:** Full US + Canadian legal corpus in one system, bilingual EN/FR, seamless cross-jurisdiction research — Harvey's coverage is US-centric despite recent expansion
- **Layer 3 — Open Ecosystem:** MCP-native architecture means any tool, any agent, any existing workflow connects. Harvey locks firms into its Vault + Workflow Builder walled garden.^[6]

Company Positioning Statement:

"LexCore is the only North American legal intelligence database your firm actually controls. Your queries never cross a US border. Your data never trains someone else's model. Your jurisdiction coverage spans the entire continent."

1.2 Strategy Team Perspective: Competitive Landscape & Market Entry

Competitive Reality Check (April 2026)

Harvey's current product: Vault (repository analysis up to 100K docs), Workflows (25,000+ custom multi-step processes), Collaboration (shared workspaces), Word Add-in. GPT-5.2 now powering all outputs. 130 of the world's largest law firms surveyed. Their ICP is BigLaw and large corporate legal departments — firms with 200+ attorneys, enterprise procurement processes, and existing Harvey relationship managers.^{[7] [1] [6]}

Harvey's documented blind spots in 2026:

- No true Canadian data sovereignty capability^[5]
- US-centric despite 110+ new sources — Canadian provincial law remains thin
- Walled garden: you must use Harvey's UI; no open MCP ecosystem
- Word Add-in as primary UX — no integration with DMS platforms at the research layer
- Pricing excludes solo practitioners and firms under 50 attorneys entirely^[8]

GTM Strategy: Three Sequential Motions

Motion 1 — Canadian Sovereignty Wedge (Phase 1):

Target Canadian law firms 20–200 attorneys. The pitch is one sentence: *"Harvey can't guarantee your client data stays out of US jurisdiction. We can."* This sells itself to any firm that has read the CLOUD Act analysis — and every Canadian law firm with US-adjacent matters has. Partner with the Canadian Bar Association, Law Society of Ontario, and CLEBC (Continuing Legal Education of BC) for credibility.^[9]

Motion 2 — Solo & Small Firm Expansion (Phase 1 concurrent):

Harvey doesn't have a sub-enterprise tier. 70% of solos now use AI tools but no affordable all-in-one legal research platform exists. LexCore's \$149/mo Solo tier is the only product in market designed specifically for this segment.^[8]

Motion 3 — Legal Tech Developer Ecosystem (Phase 2):

Publish the MCP server as open-source. Legal SaaS companies building on top of LexCore become distribution partners. Clio has 150,000+ law firm customers globally — a Clio App Marketplace integration is a direct pipeline to every firm already paying for practice management software.^[10]

Data Strategy: The Time-Compounding Citation Graph

Every document LexCore ingests enriches the citation graph. Every query teaches the relevance model. The longer LexCore runs, the more authoritative its citation chains become. Harvey processes documents for individual firms — their network effect is siloed. LexCore's citation graph compounds across the entire corpus, building an asset Harvey's architecture fundamentally cannot replicate. This is the long-term moat that accelerates with time.

1.3 CFO Perspective: Financial Model

Revenue Model

Tier	Target	Monthly Price	Query Limit	Billing Method
Solo	Solo practitioners, 1-2 attorney firms	\$149/mo	10,000 queries	Stripe metered subscription
Firm	10–200 attorney firms	\$799/mo	Unlimited	Stripe flat subscription
Enterprise	BigLaw, corporate legal, legal tech builders	\$3,500/mo	Unlimited + self-host	Manual invoicing
API/Developer	Legal tech companies, SaaS builders	\$0.02/query	Pay-as-you-go	Stripe Meters API

Cost Model

Cost Category	Phase 1	Phase 2	Phase 3
Infrastructure (GCP northamerica-northeast1)	\$154/mo	\$450/mo	\$1,100/mo
Stripe fees (2.9% + \$0.30)	Included in pricing	—	—
Legal data licensing (CanLII partnership)	\$0–\$5K one-time	—	—
CLO/legal counsel (data clearance)	\$5–10K one-time	—	—
Break-even (Solo tier only)	6 customers	—	—

Cost Category	Phase 1	Phase 2	Phase 3
Break-even (mixed)	4 Solo + 1 Firm	—	—

Note on infrastructure: GCP northamerica-northeast1 (Montréal) is selected over Azure and AWS. Physical data residency in Canada is satisfied. CLOUD Act exposure exists (Google is a US corporation) but is mitigated by Canadian incorporation of the LexCore operating entity + contractual data processing agreements. For Enterprise self-host tier, clients run entirely on their own infrastructure — full sovereignty.^{[4] [5]}

1.4 CLO Perspective: Legal, Compliance & Data Licensing

The PIPEDA Sovereignty Framework

PIPEDA does not legally require data to stay in Canada, but requires "adequate safeguards" for cross-border transfers. Quebec's Law 25 sets a higher bar — it requires a Privacy Impact Assessment (PIA) confirming the destination provides equivalent privacy protection before any cross-border transfer. Every Quebec-based law firm customer triggers Law 25 requirements.^{[11] [12]}

LexCore's Compliance Architecture:

- All tenant query data processed and stored in northamerica-northeast1 (Montréal, GCP) — physically in Canada
- Data Processing Agreement (DPA) executed with every tenant — commits LexCore to Canadian-first processing
- Quebec Law 25 PIA completed for LexCore's infrastructure configuration — available to share with Quebec-based firm clients on request
- Enterprise self-host tier: client controls all infrastructure — zero cross-border exposure of any kind
- Attorney-client privilege wall: no query content is retained after result delivery — only anonymized metrics (query count, latency, cache hit) in audit log

Data Licensing Clearance Map

Every source must receive CLO clearance before any SourceAdapter agent writes a single document to production. This check is enforced in code — SourceAdapter agents read `LEGAL_DATA_SOURCES.md` and refuse to ingest uncertified sources.^{[13] [14]}

Source	License Type	AI Training Cleared	Residency	Required Action
eCFR (US Federal)	US Public Domain	✓ Yes	N/A	None
US Code (uscode.house.gov)	US Public Domain	✓ Yes	N/A	None

Source	License Type	AI Training Cleared	Residency	Required Action
CourtListener (federal + state)	CC-BY 4.0	✔ Yes with attribution	N/A	Attribution metadata field
Justice Laws Canada (federal)	Crown Copyright	⚠ Review	Canada	Reproduction for non-commercial research: permitted. CLO to confirm AI training scope.
CanLII	ToS Restricted	⚠ Negotiate	Canada	API partnership negotiation with CanLII prior to ingest
Common Corpus Legal Commons	Open License	✔ Yes	N/A	Verify per-jurisdiction scope
PACER (US federal courts)	Paywalled	✔ via CourtListener mirror	N/A	Use CC-BY mirror exclusively
State statute portals (US)	Varies	⚠ Per-state audit	N/A	Per-state ToS audit — 5 Phase 1 states before ingest
Quebec statutes (LégisQuébec)	Crown Copyright	⚠ Review	Canada	Same as Justice Laws — CLO confirm scope

Legal Advice Liability Wall

Every LexCore output at every surface carries a mandatory non-negotiable disclaimer. This is enforced at the MCP gateway layer — it cannot be disabled by any tenant configuration.

"LexCore outputs are legal research for review by licensed counsel. They do not constitute legal advice, a legal opinion, or a solicitor-client communication. All outputs must be independently verified by a qualified legal professional before reliance."

LexDraft outputs: labeled "Draft for attorney review — not a final document"

LexAnalysis outputs: labeled "Research analysis — not a legal opinion"

All outputs: include `source_chunk_ids[]` for every legal claim made — full traceability to source material

PART II — PRODUCT DEFINITION

2.1 CPO Perspective: The Three Product Surfaces

Surface 1 — LexCore Research (*primary surface, lawyers + paralegals*)

Natural language search across full North American legal corpus. Jurisdiction filter panel. Body-of-law selector. Citation graph visualization (precedent chain as interactive tree). Export to brief (formatted PDF + citation list). Query history with saved searches. All powered by LexSearch agent via `research_task` and `search_legal` MCP tools.

Surface 2 — LexCore Monitor (*compliance officers + GC + legal ops*)

Live legislative change feed filtered by tenant's `monitor_rules`. Jurisdiction coverage map showing sync status and data freshness. Change alert timeline with auto-drafted summaries

from LexMonitor agent. Full audit log of all firm queries (board-presentable, exportable). Billing usage dashboard. Powered by LexMonitor agent via `check_updates` and `jurisdiction_summary` tools.

Surface 3 — LexCore API + MCP Gateway (*developers + enterprise + integrations*)

Full programmatic access. MCP server (primary interface). REST API (fallback for non-MCP tools). WebSocket for streaming research results. API key management portal. Developer documentation. Rate limit dashboard. Stripe billing portal. Powered directly by the MCP server layer.

LexConnect Integration Layer (*embedded in existing law firm tools*)

Clio, NetDocuments, and iManage adapters that bring LexCore research directly into the tools lawyers already use daily — no context switch required. This is the distribution multiplier.^[15] ^[16]

2.2 ICP Personas — Full Definition

Persona 1: Sarah Chen — Senior Associate, Cross-Border M&A

8-year call, Toronto BigLaw, \$200K+. Handles Canada-US corporate transactions weekly.

Daily friction: Research spanning Ontario corporate law, federal securities regulations, and SEC filings takes 4–6 hours. Harvey isn't approved by firm's IT because of CLOUD Act exposure. Westlaw costs \$800/seat/month and still has poor Canadian coverage. Manual CanLII search is slow and has no semantic understanding.

What she needs: One search across both jurisdictions simultaneously. Citation chains that show her which cases cite which. Export that formats her findings for insertion into a memorandum. Data she can tell her partner came from a sovereign, privilege-safe source.

How she buys: Doesn't pay personally. Needs to convince the firm's legal ops manager and IT security. The PIPEDA sovereignty argument + CLOUD Act avoidance is her internal sales pitch to IT.

Success metric: Cross-border research from 4 hours to 30 minutes. Zero research errors from outdated statutes. Defensible source attribution in every memo.^[17] ^[18]

Persona 2: Marcus Webb — Solo Criminal Defense, Phoenix AZ

12-year call, sole practitioner, 95% Arizona state court work.

Daily friction: Can't afford Harvey or Westlaw. Uses free resources, misses case updates, spends 2+ hours per matter on research that a \$800/seat tool would do in 10 minutes. 70% of solos now use AI but no affordable unified tool exists for their specific needs.^[8]

What he needs: Affordable AZ criminal statute search. Automated alerts when statutes he relies on change. Searchable case law without a manual Westlaw subscription. Simple enough to use in 10 minutes between client meetings.

How he buys: Solo tier, credit card, \$149/mo. Self-service. No sales process.

Success metric: Never misses a statute amendment. Case law research under 15 minutes.
Monthly cost under \$200 total for all research tools.

Persona 3: Jennifer Park — General Counsel, Vancouver Series B

Solo in-house legal, managing US + Canadian compliance for 120-person tech company.

Daily friction: Board wants AI governance documentation. Investors want compliance audit trails. She's managing BC Employment Standards Act, PIPEDA, federal securities disclosure requirements, and Delaware corporate law simultaneously with no team. One tool that spans all of it doesn't exist at a price point a startup can afford.

What she needs: Regulatory monitoring across both jurisdictions with automatic alerts. A query audit log she can export and present to the board. Bilingual EN/FR coverage for federal Canadian regulations. Hard requirement: her client query data must stay in Canada. She has read the CLOUD Act analysis. She will not sign with a US-parent vendor.^{[9] [5]}

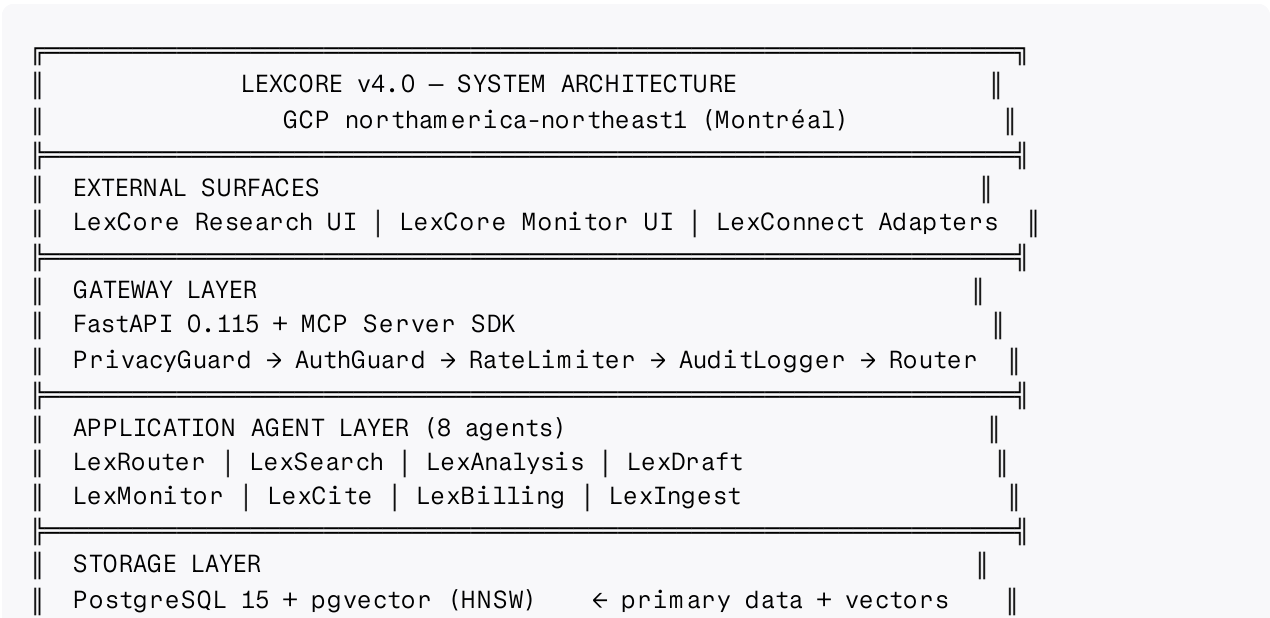
How she buys: Firm tier (\$799/mo), approved by CFO as legal ops spend. Will upgrade to Enterprise self-host if the startup reaches Series C and board mandates full data sovereignty.

Success metric: Compliance audit log delivered to board quarterly. Zero manual regulatory monitoring required. Counsel time on research < 2 hours/week.^[17]

PART III — TECHNICAL ARCHITECTURE

3.1 Chief Architect Perspective: System Architecture

The architectural principle governing every decision: **determinism above all else**. In a legal context, a system that returns different results for the same query is not a research tool — it is a liability. Every layer is designed for reproducibility, auditability, and zero silent failure.



Redis 7	← cache + Celery broker
GCS (northamerica-northeast1)	← raw doc archive, immutable
INTELLIGENCE LAYER	
Ollama (mxbai-embed-large-v1)	← local embeddings, \$0/call
Celery 5 + Beat	← ingest scheduling
Docling	← PDF/HTML extraction + OCR
OBSERVABILITY LAYER	
Prometheus + Grafana Structured JSON Logs Loki	
Correlation IDs Alert Rules PagerDuty Integration	

Stack Decisions (with rationale)

Component	Choice	Why	When to Revisit
Vector store	pgvector HNSW	Eliminates Qdrant operational complexity; handles 10M+ vectors. One less service to operate.	Phase 3 if vectors exceed 50M
Embedding model	mxbai-embed-large-v1(1024d) via Ollama	Superior to nomic-embed-text on legal domain benchmarks. Local = zero API cost, zero rate limits, zero data egress.	If legal-domain fine-tuned model ships
Database	PostgreSQL 15	ACID guarantees, RLS multi-tenancy, pgvector, full-text search in one system	Never — this is the right choice
Task queue	Celery 5 + Redis	Battle-tested, per-task retry policies, dead-letter queues, Flower monitoring	Never for this scale
Cloud region	GCP northamerica-northeast1	Physical data residency in Canada (Montréal) for PIPEDA adequacy	Enterprise tier: client infrastructure
Billing	Stripe Meters API	Cleanest metered billing in 2026, handles idempotency, first-class AI usage billing support	Never — Stripe Meters is the standard
CI/CD	GitHub Actions	Standard, agent-friendly, branch protection gates, canary deployment support	Never

3.2 Complete Database Schema

```

-- =====
-- LEXCORE v4.0 – COMPLETE PRODUCTION SCHEMA
-- PostgreSQL 15 + pgvector
-- All tables include RLS where multi-tenancy is required
-- Immutable tables: append-only, no UPDATE/DELETE
-- =====

-- ——— MULTI-TENANCY ———

CREATE TABLE tenants (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name       TEXT NOT NULL,

```



```

tier            TEXT NOT NULL CHECK (tier IN ('solo','firm','enterprise','develop
api_key_hash    TEXT UNIQUE NOT NULL,
stripe_customer_id TEXT,
stripe_sub_id   TEXT,
query_limit     INT,                -- NULL = unlimited (firm/enterprise)
self_hosted     BOOLEAN DEFAULT FALSE,
data_region     TEXT DEFAULT 'ca-montreal',
pipeda_dpa_signed BOOLEAN DEFAULT FALSE,
law25_pia_completed BOOLEAN DEFAULT FALSE,
created_at      TIMESTAMPTZ DEFAULT now(),
is_active       BOOLEAN DEFAULT TRUE
);

```

```

CREATE TABLE billing_events (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id     UUID REFERENCES tenants(id),
  billing_period TEXT NOT NULL,      -- '2026-04'
  query_count   INT DEFAULT 0,
  tier_at_billing TEXT NOT NULL,
  stripe_meter_event_id TEXT,       -- idempotency key
  amount_cents  INT,
  invoiced_at   TIMESTAMPTZ,
  created_at    TIMESTAMPTZ DEFAULT now()
);

```

-- — JURISDICTION REGISTRY —————

```

CREATE TABLE jurisdictions (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  country       TEXT NOT NULL CHECK (country IN ('US','CA')),
  level         TEXT NOT NULL CHECK (level IN ('federal','state','province','terri
  name          TEXT NOT NULL,
  name_fr       TEXT,              -- French name for CA jurisdictions
  code          TEXT UNIQUE NOT NULL, -- 'US-FED','CA-ON','US-CA','CA-FED','CA-QC'
  languages     TEXT[] DEFAULT ARRAY['en'],
  coverage_status TEXT DEFAULT 'pending'
                                CHECK (coverage_status IN ('pending','partial','complete')),
  last_full_sync TIMESTAMPTZ,
  source_urls   JSONB,            -- {statutes: url, regulations: url, case_law: u
  license_cleared BOOLEAN DEFAULT FALSE,
  phase         INT DEFAULT 2     -- 1=Phase1, 2=Phase2, 3=Phase3
);

```

-- — LEGAL DOCUMENTS —————

```

CREATE TABLE legal_documents (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  jurisdiction_id UUID NOT NULL REFERENCES jurisdictions(id),
  doc_type      TEXT NOT NULL
                CHECK (doc_type IN ('statute','regulation','case_law','template','a
  body_of_law   TEXT NOT NULL
                CHECK (body_of_law IN
                    ('civil','criminal','corporate','labor','tax','ip',
                    'constitutional','administrative','securities','privacy','family
  title        TEXT NOT NULL,
  title_fr     TEXT,

```

```

official_citation TEXT UNIQUE,
source_url       TEXT NOT NULL,
content_hash     TEXT NOT NULL,      -- SHA256 for change detection
raw_text        TEXT,                -- full extracted text
language         TEXT DEFAULT 'en' CHECK (language IN ('en','fr','bilingual')),
license_type     TEXT NOT NULL
                CHECK (license_type IN ('public_domain','cc_by','crown_copyright','
data_quality_score FLOAT CHECK (data_quality_score BETWEEN 0 AND 1),
effective_date   DATE,
repeal_date     DATE,
version         INT DEFAULT 1,
is_current      BOOLEAN DEFAULT TRUE,
is_deleted      BOOLEAN DEFAULT FALSE, -- soft delete only, never hard delete
ingested_at     TIMESTAMPTZ DEFAULT now(),
updated_at      TIMESTAMPTZ DEFAULT now()
);

```

-- — LEGAL CHUNKS (embedded text) —————

```

CREATE TABLE legal_chunks (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id UUID NOT NULL REFERENCES legal_documents(id) ON DELETE CASCADE,
  chunk_index INT NOT NULL,
  section_path TEXT,                -- 'Part II > Division 3 > s.47(2)(b)'
  chunk_text  TEXT NOT NULL,
  chunk_text_fr TEXT,
  embedding   VECTOR(1024),        -- mxbai-embed-large-v1
  token_count INT,
  quality_score FLOAT CHECK (quality_score BETWEEN 0 AND 1),
  created_at  TIMESTAMPTZ DEFAULT now(),
  UNIQUE(document_id, chunk_index)
);

```

-- — CITATION GRAPH —————

```

CREATE TABLE legal_citations (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  source_doc_id UUID NOT NULL REFERENCES legal_documents(id),
  target_doc_id UUID NOT NULL REFERENCES legal_documents(id),
  citation_text TEXT,
  citation_type TEXT NOT NULL
                CHECK (citation_type IN
                    ('references','overrules','amends','repeals','implements','uphol
confidence      FLOAT DEFAULT 1.0, -- 1.0 = extracted from structured field, <1.0 =
  UNIQUE(source_doc_id, target_doc_id, citation_type)
);

```

-- — VERSION HISTORY (IMMUTABLE – append only) —————

```

CREATE TABLE legal_updates (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id UUID NOT NULL REFERENCES legal_documents(id),
  change_type TEXT NOT NULL
                CHECK (change_type IN ('created','amended','repealed','restored','
previous_hash TEXT,
new_hash      TEXT,

```

```

diff_summary    TEXT,
affected_sections TEXT[],
detected_at     TIMESTAMPTZ DEFAULT now(),
confirmed_by    TEXT DEFAULT 'auto' -- 'auto' or human reviewer ID
);

-- — TENANT MONITORING RULES —————

CREATE TABLE monitor_rules (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id     UUID NOT NULL REFERENCES tenants(id),
  rule_name     TEXT NOT NULL,
  jurisdiction_codes TEXT[] NOT NULL,
  bodies_of_law TEXT[],
  keywords      TEXT[],
  alert_channel TEXT NOT NULL CHECK (alert_channel IN ('email','webhook','slack','i
  alert_endpoint TEXT,
  digest_frequency TEXT DEFAULT 'immediate'
                      CHECK (digest_frequency IN ('immediate','daily','weekly')),
  is_active     BOOLEAN DEFAULT TRUE,
  created_at    TIMESTAMPTZ DEFAULT now()
);

-- — CONFLICT CHECK LOG (tenant-isolated, no cross-tenant) —

CREATE TABLE conflict_checks (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id     UUID NOT NULL REFERENCES tenants(id),
  matter_name   TEXT NOT NULL,
  party_names   TEXT[] NOT NULL,
  query_match_found BOOLEAN,
  checked_at    TIMESTAMPTZ DEFAULT now()
);

-- — COMPLIANCE AUDIT LOG (IMMUTABLE – append only) —————

CREATE TABLE audit_log (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id     UUID NOT NULL REFERENCES tenants(id),
  agent_id      TEXT NOT NULL,
  tool_called   TEXT NOT NULL,
  query_fingerprint TEXT,          -- SHA256 only – NO raw query content stored
  result_doc_ids UUID[],
  result_count  INT,
  latency_ms    INT,
  cache_hit     BOOLEAN DEFAULT FALSE,
  called_at     TIMESTAMPTZ DEFAULT now()
  -- NOTE: Raw query text is NEVER stored. PIPEDA compliance requirement.
  -- Only the fingerprint (irreversible hash) is stored for deduplication/caching.
);

-- — QUERY CACHE (deterministic) —————

CREATE TABLE query_cache (
  query_fingerprint TEXT PRIMARY KEY, -- SHA256 of normalized query + params
  result            JSONB NOT NULL,

```

```

jurisdiction_scope TEXT[],
search_config      JSONB,
created_at         TIMESTAMPTZ DEFAULT now(),
expires_at         TIMESTAMPTZ NOT NULL,
hit_count          INT DEFAULT 0
);

-- — INGEST PIPELINE JOBS —————

CREATE TABLE ingest_jobs (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  jurisdiction_id    UUID NOT NULL REFERENCES jurisdictions(id),
  source_url         TEXT,
  status            TEXT NOT NULL
                    CHECK (status IN ('queued', 'running', 'complete', 'failed', 'dead_letters')),
  trigger_type       TEXT DEFAULT 'scheduled'
                    CHECK (trigger_type IN ('scheduled', 'manual', 'change_detected', 'reindex')),
  documents_processed INT DEFAULT 0,
  documents_failed   INT DEFAULT 0,
  documents_skipped  INT DEFAULT 0, -- skipped: unchanged hash
  error_detail       TEXT,
  started_at         TIMESTAMPTZ,
  completed_at       TIMESTAMPTZ,
  created_at         TIMESTAMPTZ DEFAULT now()
);

-- — EVALUATION RUNS —————

CREATE TABLE eval_runs (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  run_at            TIMESTAMPTZ DEFAULT now(),
  git_sha           TEXT NOT NULL,
  triggered_by       TEXT DEFAULT 'cicd', -- 'cicd' | 'manual' | 'scheduled'
  precision_at_5     FLOAT,
  recall_at_10       FLOAT,
  ndcg_at_10         FLOAT,
  citation_accuracy  FLOAT,
  citation_hallucination_rate FLOAT, -- target: 0.000
  total_test_cases  INT,
  passed_cases       INT,
  gate_passed        BOOLEAN NOT NULL,
  baseline_delta     JSONB, -- {metric: delta_from_baseline}
  notes             TEXT
);

-- — AGENT MANIFEST (mirrors AGENTS.md in DB) —————

CREATE TABLE agent_manifest (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  spec_version       TEXT NOT NULL,
  lane_contracts     JSONB,
  completed_outputs  JSONB,
  open_blockers      JSONB,
  last_updated       TIMESTAMPTZ DEFAULT now()
);

```

```

-- ===== INDEXES =====

-- Vector search (HNSW – high recall, fast ANN)
CREATE INDEX idx_chunks_embedding
  ON legal_chunks USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 128);

-- Full-text search (GIN)
CREATE INDEX idx_chunks_fts_en
  ON legal_chunks USING GIN (to_tsvector('english', chunk_text));

CREATE INDEX idx_chunks_fts_fr
  ON legal_chunks USING GIN (to_tsvector('french', coalesce(chunk_text_fr, '')));

-- Document lookups
CREATE INDEX idx_docs_jurisdiction
  ON legal_documents(jurisdiction_id)
  WHERE is_current = TRUE AND is_deleted = FALSE;

CREATE INDEX idx_docs_body_law
  ON legal_documents(body_of_law)
  WHERE is_current = TRUE AND is_deleted = FALSE;

CREATE INDEX idx_docs_citation
  ON legal_documents(official_citation)
  WHERE official_citation IS NOT NULL;

CREATE INDEX idx_docs_hash
  ON legal_documents(content_hash);

-- Citation graph traversal
CREATE INDEX idx_citations_source ON legal_citations(source_doc_id);
CREATE INDEX idx_citations_target ON legal_citations(target_doc_id);

-- Audit and monitoring
CREATE INDEX idx_audit_tenant_time
  ON audit_log(tenant_id, called_at DESC);

CREATE INDEX idx_updates_doc_time
  ON legal_updates(document_id, detected_at DESC);

CREATE INDEX idx_ingest_status
  ON ingest_jobs(status, jurisdiction_id);

-- Billing
CREATE INDEX idx_billing_tenant_period
  ON billing_events(tenant_id, billing_period);

-- ===== ROW LEVEL SECURITY =====

ALTER TABLE audit_log ENABLE ROW LEVEL SECURITY;
ALTER TABLE monitor_rules ENABLE ROW LEVEL SECURITY;
ALTER TABLE conflict_checks ENABLE ROW LEVEL SECURITY;
ALTER TABLE billing_events ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_audit_isolation ON audit_log

```

```

    USING (tenant_id = current_setting('app.tenant_id')::UUID);

CREATE POLICY tenant_monitor_isolation ON monitor_rules
    USING (tenant_id = current_setting('app.tenant_id')::UUID);

CREATE POLICY tenant_conflict_isolation ON conflict_checks
    USING (tenant_id = current_setting('app.tenant_id')::UUID);

CREATE POLICY tenant_billing_isolation ON billing_events
    USING (tenant_id = current_setting('app.tenant_id')::UUID);

-- ===== TRIGGERS =====

-- Auto-update updated_at on legal_documents
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN NEW.updated_at = now(); RETURN NEW; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_docs_updated_at
    BEFORE UPDATE ON legal_documents
    FOR EACH ROW EXECUTE FUNCTION update_updated_at();

```

3.3 Security Engineer Perspective: Privacy & Compliance Architecture

PrivacyGuard — Gateway Middleware

PrivacyGuard runs as the **first middleware in the request chain**, before AuthGuard and before any query touches the database. It enforces three rules on every inbound request:

1. **No PII in query payloads.** Scans query text for patterns matching names, email addresses, phone numbers, SIN/SSN, health identifiers. If detected, the request is rejected with a 400 and a `PRIVACY_VIOLATION` error code. The tenant is notified via their alert channel.
2. **Query content is never logged.** The audit log stores only the SHA256 fingerprint of the normalized query — irreversible. Raw query text is processed in memory only, never written to any persistent store.
3. **PIPEDA disclosure on first connection.** Every new tenant receives a machine-readable `X-LexCore-Privacy-Policy` header on their first authenticated request, containing the data processing summary.

Conflict of Interest Isolation

Cross-tenant analytics are **permanently and architecturally disabled** — not a configuration flag, a hardcoded prohibition enforced at the query layer. The `conflict_check` MCP tool operates only within a single tenant's own `audit_log` — it cannot access any other tenant's data even in aggregate form.

Prometheus metrics that emit query-level data use **anonymized labels only**: `{tenant_tier, jurisdiction_category, cache_hit}` — never query content, never tenant name, never

document identifiers.

3.4 MCP Server — Complete Tool Contracts

Eight tools. `get_capabilities` is called first by every agent on initialization to self-orient. `research_task` is the macro action for complex work. Everything else serves specific workflow needs. [\[19\]](#) [\[20\]](#)

```
# lexcore_mcp_server.py — FastAPI 0.115 + MCP Server SDK
# GCP northamerica-northeast1 | Python 3.12 | All tools below

DISCLAIMER = (
    "LexCore outputs are legal research for review by licensed counsel. "
    "They do not constitute legal advice, a legal opinion, or a solicitor-client "
    "communication. All outputs must be independently verified by a qualified "
    "legal professional before reliance."
)

# — TOOL 1: CAPABILITY MANIFEST —————
@tool(description=
    "Returns full capability manifest: jurisdictions, coverage status, bodies of law,
    "data freshness. ALWAYS call this first on initialization to orient before querying
    async def get_capabilities() -> dict:
        # Returns: schema_version, total_documents, jurisdictions[], bodies_of_law[],
        #           doc_types[], last_system_update, api_version, data_region
        ...

# — TOOL 2: MACRO RESEARCH TASK —————
@tool(description=
    "Execute a full legal research task from a natural language question. "
    "Decomposes question into sub-queries, searches in parallel, builds citation chain
    "returns structured research report with confidence score and source traceability.
    "PRIMARY tool for complex research. Use search_legal for precise targeted queries
    async def research_task(
        question: str,
        jurisdictions: list[str] | None = None,
        body_of_law: str | None = None,
        depth: str = "standard" # 'quick' | 'standard' | 'deep'
    ) -> dict:
        # Returns: question, sub_queries[], primary_sources[], supporting_sources[],
        #           citation_chain{}, jurisdiction_gaps[], confidence_score,
        #           data_freshness{per_jurisdiction}, disclaimer, source_chunk_ids[]
        ...

# — TOOL 3: PRECISE SEARCH —————
@tool(description=
    "Hybrid semantic + keyword search. Returns ranked legal chunks with confidence "
    "scores, section paths, and data freshness. For complex questions use research_tas
    "Configurable semantic/keyword weighting via search_config.")
    async def search_legal(
        query: str,
        jurisdiction: str | None = None, # ISO code: 'US-FED', 'CA-ON', 'US-CA'
        body_of_law: str | None = None,
        doc_type: str | None = None,
```

```

    top_k: int = 10,
    language: str = "en",          # 'en' | 'fr' | 'both'
    search_config: dict | None = None # {"semantic_weight": 0.7, "keyword_weight":
) -> dict:
    # Returns: results[], cache_hit, query_fingerprint, result_count,
    #         search_config_used, data_freshness{}, disclaimer
    ...

# — TOOL 4: FULL DOCUMENT RETRIEVAL —————
@tool(description=
    "Retrieve a full legal document by ID or official citation. "
    "Returns complete text, all chunks with section paths, citation list, and version r
async def get_document(
    doc_id: str | None = None,
    citation: str | None = None,
    include_chunks: bool = True,
    include_citations: bool = True
) -> dict: ...

# — TOOL 5: CITATION GRAPH —————
@tool(description=
    "Get the citation graph for a document. Traverse what it references and what "
    "references it. Returns chain-of-authority with overruled case flags. "
    "direction: 'upstream' | 'downstream' | 'both'. depth: 1-3.")
async def get_citations(
    doc_id: str,
    direction: str = "both",
    depth: int = 2,
    include_overruled_flags: bool = True
) -> dict: ...

# — TOOL 6: LEGISLATIVE MONITORING —————
@tool(description=
    "Check for all legislative changes in a jurisdiction since a given date. "
    "Returns change list with diff summaries, change types, and affected sections. "
    "Powers LexMonitor agent and tenant alert rules.")
async def check_updates(
    jurisdiction: str,
    since_date: str,          # ISO date: '2026-01-01'
    body_of_law: str | None = None,
    include_diff_summary: bool = True
) -> dict: ...

# — TOOL 7: JURISDICTION SUMMARY —————
@tool(description=
    "Get a full summary of a jurisdiction: document count, coverage percentage, "
    "bodies of law covered, data freshness, recent changes, and sync status.")
async def jurisdiction_summary(
    jurisdiction_code: str
) -> dict: ...

# — TOOL 8: CONFLICT CHECK —————
@tool(description=
    "Scan this tenant's own query history for any prior research involving "
    "the specified parties or matter name. For internal conflict-of-interest "
    "screening only. Never accesses other tenants' data.")

```



```

async def conflict_check(
    matter_name: str,
    party_names: list[str],
    lookback_days: int = 365
) -> dict: ...

```

PART IV — DATA INTELLIGENCE LAYER

4.1 IngestArchitect Perspective: Pipeline Architecture

The ingest pipeline is fully autonomous. The only time a human is involved is when a document fails quality scoring below 0.7 and requires review — all other processing is zero-touch.^[20]

```

Source URL → SourceAdapter → Docling Parser → License Validator
           → Quality Scorer → Chunker → EmbeddingWorker → DB Writer
           → Change Detector → CitationExtractor → Alert Trigger
               ↓ on failure
Dead-Letter Queue → SRE Alert → Human Review Queue

```

Celery Task Architecture

```

# Celery Beat Schedule – per source, per cadence
CELERY_BEAT_SCHEDULE = {
    # Phase 1 sources
    "ingest-canLII-federal": {"task": "ingest.run", "schedule": crontab(hour=2, mi
                                "args": ["CA-FED", "canLII"]},
    "ingest-canLII-ON": {"task": "ingest.run", "schedule": crontab(hour=2, mi
                                "args": ["CA-ON", "canLII"]},
    "ingest-canLII-BC": {"task": "ingest.run", "schedule": crontab(hour=3, mi
                                "args": ["CA-BC", "canLII"]},
    "ingest-canLII-AB": {"task": "ingest.run", "schedule": crontab(hour=3, mi
                                "args": ["CA-AB", "canLII"]},
    "ingest-justice-laws": {"task": "ingest.run", "schedule": crontab(hour=1, mi
                                "args": ["CA-FED", "justice_laws"]},
    # Phase 2 sources (added after Phase 1 gate)
    "ingest-eCFR": {"task": "ingest.run", "schedule": crontab(hour=4, mi
                                "args": ["US-FED", "ecfr"]},
    "ingest-courtlistener": {"task": "ingest.run", "schedule": crontab(hour=4, m
                                "args": ["US-FED", "courtlistener"]},
}

# Task retry policy (all ingest tasks)
RETRY_POLICY = {
    "max_retries": 3,
    "countdown": 300,          # 5 min initial backoff
    "backoff": "exponential" # 5m → 25m → 125m
}

```

```
# Dead-letter: after 3 retries, write to ingest_jobs with status='dead_letter'
# and fire SRE alert. Never silently discard.
```

Source Adapter Registry

Each adapter is a self-contained Python class with a standard interface. The SourceAdapter horde agents write one adapter per source. New sources in Phase 2/3 require only a new adapter class — no changes to the pipeline architecture.

```
class SourceAdapterBase(ABC):
    """Standard interface all source adapters must implement."""

    @abstractmethod
    async def fetch_document_list(self) -> list[dict]:
        """Return list of {url, citation, expected_hash} for jurisdiction."""

    @abstractmethod
    async def fetch_document(self, url: str) -> RawDocument:
        """Fetch and parse one document. Return RawDocument with all fields."""

    @abstractmethod
    def check_license(self) -> bool:
        """Verify LEGAL_DATA_SOURCES.md shows CLO clearance. Refuse if not."""

    @abstractmethod
    def detect_format_change(self, response: httpx.Response) -> bool:
        """Detect if source has changed format. Trigger dead-letter if True."""
```

Chunking Strategy — Hierarchical

```
# Hierarchy: Act → Part → Division → Section → Subsection → Paragraph
# Max chunk: 512 tokens
# Overlap: 50 tokens between adjacent chunks (preserves context at boundaries)
# Section path: always preserved – 'Part II > Division 3 > s.47(2)(b) '
# Quality gate: reject chunk if quality_score < 0.7 (OCR confidence too low)
# Bilingual: EN and FR chunks stored in parallel (chunk_text + chunk_text_fr)
```

PART V — APPLICATION AGENT DIRECTORY

Eight LexCore application agents. Each owns its entire functional domain end-to-end. No agent writes directly to the database — all reads/writes go through the internal service layer. [\[21\]](#) [\[20\]](#)

Agent	Domain	Primary MCP Tools	Autonomy	Trigger
LexRouter	Intent classification, agent dispatch, capability orientation	get_capabilities+all	100%	Every inbound query

Agent	Domain	Primary MCP Tools	Autonomy	Trigger
LexSearch	Complete legal research — structured report, ranked results, citation support	research_task, search_legal, get_document	100%	On-demand
LexAnalysis	Cross-jurisdiction comparison, conflict identification, trend analysis. Stores analysis back to LexCore as doc_type='analysis'	research_task, get_citations, search_legal	85%	On-demand
LexDraft	Document generation from legal templates + jurisdiction context. All claims grounded to chunk_id. Draft label always present.	search_legal, get_document, get_capabilities	80%	On-demand
LexMonitor	Watches all tenant monitor_rules. Detects changes. Auto-drafts summaries. Pushes alerts per alert_channel.	check_updates, jurisdiction_summary	90%	Every 6h (Celery Beat)
LexCite	Citation verification, precedent chain analysis, overruled case detection, chain-of-authority summaries	get_citations, get_document	100%	On-demand
LexBilling	Query count tracking, Stripe Meters event reporting, tier enforcement, usage dashboards	Internal + Stripe API	100%	Per-query (async) + monthly
LexIngest	Full pipeline orchestration: scrape → parse → chunk → embed → store. Self-heals on format changes.	Internal pipeline	95%	Celery Beat schedule

Citation Grounding Contract (LexDraft + LexAnalysis)

Every sentence in LexDraft or LexAnalysis output that makes a legal claim **must reference a specific chunk_id** from the LexCore database. This is enforced as a structured output schema — the LLM prompt requires a `claims[]` array where each claim has `text` and `source_chunk_id`. Any output that fails to provide `source_chunk_id` for a legal claim is rejected by the gateway before delivery. EvalRunner tracks `citation_hallucination_rate` — target is permanently 0.000.^[22]

LexBilling — Stripe Meters Integration

```
# Stripe Meters API (2026 standard – replaces legacy usage records) [web:116][web:12]
import stripe

stripe.api_key = settings.STRIPE_SECRET_KEY
```

```

class LexBilling:
    METER_ID = "meter_lexcore_queries" # created once in Stripe dashboard

    async def record_query(self, tenant_id: UUID, tool_called: str):
        tenant = await get_tenant(tenant_id)

        # Solo tier: enforce hard cap at 10,000 queries/month
        if tenant.tier == 'solo':
            count = await get_monthly_query_count(tenant_id)
            if count >= 10000:
                raise HTTPException(429, detail={
                    "error": "QUERY_LIMIT_EXCEEDED",
                    "message": "Solo tier limit of 10,000 queries/month reached.",
                    "upgrade_url": "https://lexcore.ca/upgrade"
                })
            if count >= 8000: # 80% warning
                await send_usage_warning(tenant_id, count)

        # Report usage event to Stripe Meters (idempotent)
        if tenant.stripe_sub_id and tenant.tier == 'developer':
            stripe.billing.meter_events.create(
                event_name="lexcore_query",
                payload={
                    "stripe_customer_id": tenant.stripe_customer_id,
                    "value": "1"
                },
                identifier=f"{tenant_id}:{tool_called}:{uuid4()}" # idempotency key
            )

        # Record locally for dashboard + audit
        await db.execute("""
            INSERT INTO billing_events (tenant_id, billing_period, query_count, tier)
            VALUES ($1, to_char(now(), 'YYYY-MM'), 1, $2)
            ON CONFLICT (tenant_id, billing_period)
            DO UPDATE SET query_count = billing_events.query_count + 1
            """, tenant_id, tenant.tier)

```

PART VI — LEXCONNECT INTEGRATION LAYER

LexConnect Adapters — The Distribution Multiplier

Law firms already live in Clio, NetDocuments, and iManage. If research doesn't happen inside those tools, lawyers will not change their workflow to use LexCore. LexConnect brings LexCore's intelligence directly into the DMS environment via MCP adapters.^[16] ^[15]

```

Law Firm DMS (Clio/NetDocuments/iManage)
→ LexConnect Adapter (MCP client)
→ LexCore MCP Gateway
→ LexSearch / LexDraft / LexCite agents
→ Results pushed back into matter workspace

```

`lexconnect-clio` — Reads active matter context from Clio API. Passes matter area + jurisdiction to `LexRouter`. Returns research results as a matter note attached to the correct file. Appears natively in Clio's timeline view. Distributed via Clio App Marketplace (150,000+ firm access).^[10]

`lexconnect-netdocuments` — Integrates via NetDocuments `ndConnect` program. Pulls document context from open matter workspace. Pushes research results directly into the matter document library as a research memo.^[16]

`lexconnect-imanage` — Reads/writes iManage Work matter workspaces via REST API. Delivers research as Work 10 documents directly into the correct matter folder. Clio and iManage now have a direct integration — LexConnect sits between them as the intelligence layer.^[15]

PART VII — QUALITY, TESTING & EVALUATION

QA Lead + AI Evals Engineer Perspective

Following the Gauntlet 5-Stage Framework — the only proven pattern for production-grade legal AI evaluation.^[23]

Stage 1 — Golden Set Construction (50 cases minimum)

`TestWriter` agent builds the harness. Legal SME (one per ICP persona) provides the annotations. This is the **only required human step** in the entire build.

Distribution of 50 golden cases:

- 12 cases: Single Canadian jurisdiction — statute lookup (CA-ON corporate, CA-FED criminal, CA-BC employment)
- 12 cases: Single US jurisdiction — statute + case law (US-FED criminal, US-CA corporate, US-NY securities)
- 10 cases: Cross-border US+CA comparison (the core ICP value proposition)
- 8 cases: Bilingual EN/FR federal Canadian regulations
- 8 cases: Legislative change detection (known amendments, known repeals)

Each case includes: `input_query`, `expected_top_3_doc_ids[]`, `expected_citation_chain{}`, `minimum_relevance_score`, `jurisdiction_scope[]`, `human_annotation` (attorney-validated summary of correct answer).

Stage 2 — Human Annotation

Three attorneys annotate: one cross-border M&A associate (Sarah Chen persona), one solo criminal defense (Marcus Webb persona), one in-house GC (Jennifer Park persona). Each annotates their relevant case subset. Annotations stored in `tests/golden_set.json`.

Stage 3 — LLM-as-Judge Calibration

EvalRunner trains the judge prompt against annotations. Calibration target: judge score within ± 0.5 of human score across all 50 cases. Judge prompt is stored in `evals/judge_prompt.md` and versioned with the codebase.

Stage 4 — Automated Metrics (per deployment)

Metric	Target	Block Threshold	Description
Precision@5	≥ 0.85	< 0.80	Top 5 results all relevant
Recall@10	≥ 0.80	< 0.75	Known-relevant docs in top 10
NDCG@10	≥ 0.78	< 0.72	Ranking quality
Citation Accuracy	100%	$< 100\%$	All cited docs exist and are current
Citation Hallucination Rate	0.000	> 0.000	Any hallucinated claim blocks deploy
Data Freshness	100%	$< 95\%$	Docs within expected sync window
P95 Latency (search_legal)	$< 400\text{ms}$	$> 600\text{ms}$	Per-tool SLA
P95 Latency (research_task)	$< 3\text{s}$	$> 5\text{s}$	Per-tool SLA

Stage 5 — Regression Gate

Every deployment runs the complete golden set before any traffic is served. Any metric that drops more than 5% from its established baseline blocks the deployment. No exceptions. Canary runs 10% traffic for 48 hours before full promotion. Full rollback capability in one command.

PART VIII — SRE & OPERATIONS

SRE Perspective: Reliability Targets & Incident Runbooks

SLA: 99.9% uptime on MCP gateway (8.7 hours maximum downtime/year)

Alert thresholds:

- Error rate $> 0.5\%$ → PagerDuty immediate
- P95 latency $> 400\text{ms}$ (search_legal) → auto-scale + alert
- P95 latency $> 3\text{s}$ (research_task) → alert + investigation
- Ingest job failed → SOP-001 (source format change) or SOP-002 (transient failure)
- Dead-letter queue depth > 50 documents → SRE alert + human review queue

Incident Runbooks:

INC-001 – Government source URL/format changed:

→ SourceAdapter triggers format change detection → source automatically paused

(status='paused' in ingest_jobs) → SRE alert fires → Data Engineer reviews → adapter updated → manual ingest run → source resumes. No data corruption. No silent failure.

INC-002 – MCP gateway latency spike:

→ Cloud Run auto-scales (max 10 instances) → alert fires → SRE reviews slow query log → pgvector index refresh if needed → connection pool check → root cause documented in incidents/ folder

INC-003 – Repealed law returned in search results:

→ Immediate cache invalidation for affected content_hash → is_current = FALSE set on document → legal_updates record appended → affected tenant notification via monitor_rules alert → EvalRunner re-run against golden set → root cause in citation graph or ingest pipeline identified

INC-004 – Citation hallucination detected in LexDraft/LexAnalysis output:

→ Immediate suspension of affected agent until root cause identified → tenant notified → CriticAgent review of last 24h outputs from that agent → grounding contract validation → citation_hallucination_rate metric reviewed → fix deployed + full eval run before agent re-enabled

INC-005 – PIPEDA/privacy concern raised by tenant:

→ Immediate escalation to CLO + CEO → audit log review (fingerprints only — no raw query data stored) → Privacy Impact Assessment review → Law 25 PIA check for Quebec tenants → response within 24 hours

PART IX — THE AGENTIC BUILD HORDE

Full Roster: 26 Agents, 7 Lanes

The Horde builds LexCore from spec to production. 26 specialist agents. Sync-Lane Protocol. Gate-controlled promotion. No human timelines — execution measured in Agent Execution Units (AEUs). [\[24\]](#) [\[25\]](#) [\[26\]](#)

Lane 0 — Command & Control (3 agents)

HordeMaster

Every function: Full spec decomposition → dependency graph → lane contract authoring → agent assignment → gate monitoring → blocker resolution → branch promotion → integration orchestration → final production promote decision. Never writes application code. The only agent that communicates across all lanes.

ContextKeeper

Every function: AGENTS.md initialization → schema version registry → approved contract log → completed output registry → open blocker log → agent state tracking → manifest DB write (agent_manifest table) → manifest refresh after every gate pass.

CriticAgent

Every function: Output receipt from every agent → contract compliance check → code quality

review (does it meet the spec contract?) → test coverage verification → schema alignment check → linter output review → structured revision request on fail → approval record + AGENTS.md update on pass. Reviews every agent output before promotion.

Lane 1 — Foundation (4 agents, fully parallel)

SchemaArchitect

Every function: Full Alembic migration set (all tables from v4.0 schema above) → migration ordering (dependency-safe) → seed data for all Phase 1 jurisdictions → RLS policy scripts → index creation scripts → alembic upgrade head self-test on fresh Postgres → schema contract publication to AGENTS.md (this contract is the dependency for all Lane 2+ agents).

Gate: Clean migration run. All tables, indexes, RLS policies verified.

DockerCompose

Every function: docker-compose.yml (

✱✱

1. <https://www.harvey.ai/blog/the-brief-january-2026>
2. <https://www.harvey.ai/blog/the-brief-april-2026>
3. <https://www.harvey.ai/blog/the-brief-march-2026>
4. <https://morleymedia.dev/blog/ai-data-residency-canadian-regulatory-requirements>
5. <https://augureai.ca/blog/which-ai-support-vendors-let-us-restrict-data-residency-to-c>
6. <https://www.spellbook.legal/briefs/harvey-vs-igidis>
7. <https://www.harvey.ai/blog/2026-skills-survey-where-legal-ai-is-working>
8. <https://abovethelaw.com/2025/12/five-new-ai-powered-business-models-for-solos-and-smalls-in-2026/>
9. <https://www.opsintel.io/blog/ai-compliance-for-canadian-law-firms-2026/>
10. <https://www.clio.com/mid-sized-law-firms/solutions/integrations/>
11. <https://learn.microsoft.com/en-us/azure/compliance/offerings/offering-canada-privacy-laws>
12. <https://digitalstaff.ca/ai-governance/data-residency-compliance>
13. <https://www.reuters.com/legal/legalindustry/copyright-law-2025-courts-begin-draw-lines-around-ai-training-piracy-market-harm--pracin-2026-03-16/>
14. <https://www.osler.com/en/insights/reports/2025-legal-outlook/ai-training-on-trial-the-next-legal-frontier-in-copyright-law/>
15. <https://imanager.com/resources/resource-center/news/newly-available-app-seamlessly-connects-imanager-and-clio/>
16. <https://www.prnewswire.com/news-releases/vincent-by-clio-joins-netdocuments-ndconnect-program-to-deliver-secure-global-legal-ai-research-302617773.html>
17. <https://www.latentbridge.com/insights/the-most-important-ai-trends-for-law-firms-in-2026-what-will-actually-change-in-legal-operations-and-client-delivery>
18. <https://www.thomsonreuters.com/en-us/posts/legal/legalweek-2026-firm-client-divide/>

19. <https://developers.redhat.com/articles/2026/01/08/building-effective-ai-agents-mcp>
20. <https://www.the-ai-corner.com/p/andrej-karpathy-ai-workflow-shift-agentic-era-2026>
21. <https://www.linkedin.com/pulse/agentic-shift-rewriting-organization-bassel-haidar-zcmze>
22. <https://ogletree.com/insights-resources/blog-posts/the-intersection-of-ai-and-attorney-client-privilege-a-cautionary-tale/>
23. <https://www.tenex.co/playbooks/how-to-test-your-ai-5-stage-framework-for-evaluating-agents-llms>
24. <https://news.agentcommunity.org/issues/2026-02-11-sovereign-swarms-and>
25. <https://techcommunity.microsoft.com/blog/appsonazureblog/the-swarm-diaries-what-happens-when-you-let-ai-agents-loose-on-a-codebase/4501393>
26. <https://aiautomationglobal.com/blog/agentic-coding-revolution-multi-agent-teams-2026>
27. <https://www.tenex.co/playbooks/one-person-growth-team>
28. <https://www.tenex.co/playbooks/guide-to-vibe-coding-using-gemini-3>
29. <https://www.tenex.co/playbooks/how-to-use-claude-code-101>
30. <https://www.tenex.co/playbooks/how-to-ship-software-without-touching-your-keyboard>
31. <https://www.tenex.co/ai-engineering>
32. <https://stackconsulting.pro>
33. <https://www.tenex.co/playbooks/show-then-tell-how-high-fidelity-prototypes-collapse-product-alignment>
34. https://www.tenex.co/playbooks/5-step-guide-for-creating-hollywood-caliber-ads-using-ai?_bhlid=6f5cfd37988910195f9342eaaf7fcf046edb040
35. https://www.tenex.co/playbooks/5-step-guide-for-creating-hollywood-caliber-ads-using-ai?_bhlid=454c0a551553e074673964599128f1e1fa0a29a
36. <https://www.definely.com/blogs/harvey-ai-competitors>
37. https://www.linkedin.com/posts/geniusworks_harveycould-transform-the-legal-world-activity-7442547726011027458-hq2d
38. <https://starterpick.com/blog/stripe-metered-billing-saas-boilerplate-2026>
39. <https://www.nxcode.io/resources/news/build-saas-stripe-payments-opencode-2026>
40. <https://hunchbite.com/guides/stripe-billing-saas>
41. <https://www.harvey.ai>